

```
create database assignment60;
```

```
use assignment60;
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10,2),  
    manager_id INT,  
    hire_year INT  
);
```

```
INSERT INTO employees
```

```
(emp_id, emp_name, department, salary, manager_id, hire_year)
```

```
VALUES
```

```
(101, 'Alice Johnson', 'IT', 72000, NULL, 2018),
```

```
(102, 'Bob Smith', 'IT', 65000, 101, 2019),
```

```
(103, 'Carol White', 'HR', 58000, NULL, 2017),
```

```
(104, 'David Brown', 'HR', 54000, 103, 2020),
```

```
(105, 'Eva Green', 'Finance', 80000, NULL, 2016),
```

```
(106, 'Frank Black', 'Finance', 75000, 105, 2018),
```

```
(107, 'Grace Lee', 'IT', 68000, 101, 2021),
```

```
(108, 'Henry Wilson', 'Marketing', 60000, NULL, 2017),
```

```
(109, 'Ivy Turner', 'Marketing', 56000, 108, 2022),
```

```
(110, 'Jack Davis', 'Finance', 70000, 105, 2019),
```

```
(111, 'Karen Moore', 'IT', 63000, 101, 2020),
```

```
(112, 'Leo Harris', 'HR', 50000, 103, 2021),
```

```
(113, 'Mia Clark', 'Marketing', 62000, 108, 2018),
```

```
(114, 'Nate Lewis', 'Finance', 78000, 105, 2019),
```

```
(115, 'Olivia Walker', 'IT', 71000, 101, 2017),
```

```
(116, 'Paul Hall', 'HR', 53000, 103, 2020),  
(117, 'Quinn Young', 'Marketing', 59000, 108, 2021),  
(118, 'Rachel King', 'Finance', 82000, 105, 2016),  
(119, 'Sam Scott', 'IT', 66000, 101, 2022),  
(120, 'Tina Adams', 'HR', 55000, 103, 2019);
```

-- 2. ORDERS TABLE

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    amount DECIMAL(10,2),  
    order_date DATE,  
    status VARCHAR(20)  
);  
  
INSERT INTO orders  
(order_id, customer_id, product_id, amount, order_date, status)  
VALUES  
(1001, 201, 301, 1500, '2024-01-05', 'Delivered'),  
(1002, 202, 302, 2300, '2024-01-12', 'Delivered'),  
(1003, 203, 303, 800, '2024-02-03', 'Delivered'),  
(1004, 201, 304, 3200, '2024-02-14', 'Delivered'),  
(1005, 204, 301, 1100, '2024-03-01', 'Pending'),  
(1006, 205, 302, 4500, '2024-03-15', 'Delivered'),  
(1007, 202, 303, 950, '2024-03-22', 'Cancelled'),  
(1008, 206, 304, 2100, '2024-04-05', 'Delivered'),  
(1009, 203, 301, 1750, '2024-04-18', 'Pending'),  
(1010, 207, 302, 3300, '2024-04-30', 'Delivered'),
```

```
(1011, 204, 303, 680, '2024-05-10', 'Cancelled'),  
(1012, 208, 304, 2900, '2024-05-22', 'Delivered'),  
(1013, 205, 301, 1200, '2024-06-08', 'Delivered'),  
(1014, 201, 302, 2700, '2024-06-15', 'Pending'),  
(1015, 209, 303, 500, '2024-06-28', 'Delivered'),  
(1016, 206, 304, 3800, '2024-07-07', 'Delivered'),  
(1017, 207, 301, 1600, '2024-07-19', 'Delivered'),  
(1018, 210, 302, 4100, '2024-07-25', 'Delivered'),  
(1019, 208, 303, 720, '2024-08-02', 'Cancelled'),  
(1020, 209, 304, 2200, '2024-08-14', 'Delivered');
```

-- 3. PRODUCTS TABLE

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(50),  
    category VARCHAR(50),  
    price DECIMAL(10,2),  
    stock_qty INT  
);
```

```
INSERT INTO products  
(product_id, product_name, category, price, stock_qty)  
VALUES  
(301, 'Laptop', 'Electronics', 55000, 30),  
(302, 'Smartphone', 'Electronics', 22000, 80),  
(303, 'Headphones', 'Electronics', 3500, 120),  
(304, 'Monitor', 'Electronics', 18000, 45),
```

(305, 'Keyboard', 'Accessories', 1500, 200),
(306, 'Mouse', 'Accessories', 900, 250),
(307, 'Desk Chair', 'Furniture', 8500, 40),
(308, 'Standing Desk', 'Furniture', 22000, 15),
(309, 'Webcam', 'Electronics', 4200, 70),
(310, 'USB Hub', 'Accessories', 1200, 180),
(311, 'Notebook', 'Stationery', 150, 500),
(312, 'Pen Set', 'Stationery', 250, 600),
(313, 'Printer', 'Electronics', 12000, 25),
(314, 'Scanner', 'Electronics', 9000, 20),
(315, 'External SSD', 'Electronics', 7500, 60),
(316, 'Lamp', 'Furniture', 2200, 90),
(317, 'Whiteboard', 'Office Supplies', 3800, 35),
(318, 'Cable Organiser', 'Accessories', 600, 300),
(319, 'Power Bank', 'Electronics', 2800, 100),
(320, 'Smart Watch', 'Electronics', 15000, 55);

-- 4. CUSTOMERS TABLE

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    city VARCHAR(50),  
    country VARCHAR(50),  
    join_year INT  
);
```

INSERT INTO customers

(customer_id, customer_name, city, country, join_year)

VALUES

(201, 'Aarav Mehta', 'Mumbai', 'India', 2021),
(202, 'Priya Sharma', 'Delhi', 'India', 2020),
(203, 'Rohan Gupta', 'Pune', 'India', 2022),
(204, 'Sunita Patel', 'Ahmedabad', 'India', 2021),
(205, 'Vikram Rao', 'Bangalore', 'India', 2019),
(206, 'Neha Singh', 'Chennai', 'India', 2023),
(207, 'Arjun Kumar', 'Hyderabad', 'India', 2020),
(208, 'Deepa Nair', 'Kochi', 'India', 2022),
(209, 'Sanjay Joshi', 'Jaipur', 'India', 2021),
(210, 'Meera Iyer', 'Coimbatore', 'India', 2023),
(211, 'Ravi Verma', 'Lucknow', 'India', 2020),
(212, 'Anita Das', 'Kolkata', 'India', 2019),
(213, 'Kiran Reddy', 'Vizag', 'India', 2022),
(214, 'Pooja Shah', 'Surat', 'India', 2021),
(215, 'Amit Bose', 'Nagpur', 'India', 2023),
(216, 'Tara Menon', 'Trivandrum', 'India', 2020),
(217, 'Nikhil Kulkarni', 'Nashik', 'India', 2022),
(218, 'Swati Chatterjee', 'Bhopal', 'India', 2019),
(219, 'Rahul Pandey', 'Patna', 'India', 2021),
(220, 'Divya Tiwari', 'Indore', 'India', 2023);

-- 5. DEPARTMENTS TABLE

CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50),

```
budget DECIMAL(12,2),  
location VARCHAR(50),  
head_id INT  
);
```

```
INSERT INTO departments  
(dept_id, dept_name, budget, location, head_id)  
VALUES  
(1, 'IT', 500000, 'Bangalore', 101),  
(2, 'HR', 200000, 'Mumbai', 103),  
(3, 'Finance', 600000, 'Delhi', 105),  
(4, 'Marketing', 300000, 'Chennai', 108);
```

-- Q1. Find the names, departments, and salaries of all employees whose salary is greater than the average salary of all employees in the company. The company-wide average salary should be computed inside a subquery.

```
select emp_name, department, salary from employees  
where salary > (select avg(salary) from employees)  
order by salary desc;
```

-- Q2. Display the emp_id, emp_name, department, and salary of the single highest-paid employee in the entire company. Use a subquery that returns the maximum salary, and filter the main query to match it.

```
select emp_id, emp_name, department, salary from employees  
where salary = (select MAX(salary) from employees);
```

-- Q3. Display the emp_id, emp_name, department, and salary of the employee with the lowest salary in the entire company.

```
select emp_id, emp_name, department, salary from employees
where salary = (select min(salary) from employees);
```

-- Q4. Find all employees (other than Rachel King herself) who were hired in the same year as Rachel King. Return their emp_name, department, and hire_year. Use a single-row subquery to find Rachel King's hire year.

```
select emp_name, department, hire_year from employees
where hire_year = (select hire_year from employees
where emp_name = 'rachel king')
and emp_name <> 'rachel king';
```

-- Q5. List all products whose price is greater than the average price across all products in the entire products table. Show product_name, category, and price, ordered by price descending.

```
select product_name, category, price from products
where price > (select avg(price) from products)
order by price desc;
```

-- Q6. Retrieve the product_id, product_name, category, and price of the single most expensive product in the entire products table.

```
select product_id, product_name, category, price from products
where price = (select max(price) from products);
```

-- Q7. Retrieve the product_id, product_name, category, and price of the cheapest product in the entire products table.

```
select product_id, product_name, category, price from products
```

where price = (select min(price) from products);

-- Q8. List all orders whose amount is greater than the average order amount across all orders in the orders table. Show order_id, customer_id, and amount, ordered by amount descending.

```
select order_id, customer_id, amount from orders
where amount > (select avg(amount) from orders)
ORDER BY amount DESC;
```

-- Q9. Find the order_id, customer_id, amount, and order_date of the single most expensive order ever placed (the order with the highest amount in the entire orders table).

```
select order_id, customer_id, amount, order_date from orders
where amount = (select max(amount) from orders);
```

-- Q10. Using the IN operator with a subquery, find all customers who have placed at least one order. Return customer_id, customer_name, and city, ordered by customer_id.

```
select customer_id, customer_name, city from customers
where customer_id in (select customer_id from orders)
order by customer_id;
```

-- Q11. Using the NOT IN operator with a subquery, find all customers who have NEVER placed any order. Return customer_id, customer_name, and city, ordered by customer_id.

```
select customer_id, customer_name, city from customers
where customer_id not in (select customer_id from orders)
order by customer_id;
```

-- Q12. Using the IN operator with a subquery on the orders table, find all products that appear in at least one order. Show product_id, product_name, category, and price.

```
select product_id, product_name, category, price from products
```



```
where product_id in (select product_id from orders);
```

-- Q13. Using the NOT IN operator, find all products that have NEVER appeared in any order. Return product_id, product_name, category, and price.

```
select product_id, product_name, category, price from products
where product_id not in (select product_id from orders);
```

```
/*
```

Q14. Using an IN subquery against the departments table, retrieve the emp_name and salary of all employees who

belong to the Finance department. Order by salary descending. (Do not hardcode the department name in the

WHERE clause of the outer query — use a subquery to fetch it.)*/

```
select emp_name, salary from employees
where department in (select dept_name from departments
where dept_name = 'Finance')
order by salary desc;
```

```
/*
```

-- Q15. Find all employees whose salary is less than the minimum salary of any employee in the Finance department.

Return emp_name, department, and salary, ordered by salary descending. (Use a single-row subquery returning MIN

salary of Finance.)*/

```
select emp_name, department, salary from employees
where salary < (select min(salary) from employees
where department = 'finance')
order by salary desc;
```

-- Q16. Find the department with the highest budget from the departments table. Return dept_name, budget, and location. Use a single-row subquery that returns MAX(budget).

```
select dept_name, budget, location from departments
where budget = (select max(budget) from departments);
```

-- Q17. Using a subquery with GROUP BY and HAVING, find all customers who have placed exactly 1 order. Return customer_id, customer_name, and city.

```
select customer_id, customer_name, city from customers
where customer_id in (select customer_id from orders
group by customer_id
having count(*) = 1);
```

-- Q18. Using a subquery with GROUP BY and HAVING, find all customers who have placed 2 or more orders. Return customer_id, customer_name, and city, ordered by customer_id.

```
select customer_id, customer_name, city from customers
where customer_id in (select customer_id from orders
group by customer_id
having count(*) >= 2)
ORDER BY customer_id;
```

/*

-- Q19. Find all products whose price is greater than the maximum price of any product in the 'Accessories' category.

Use a scalar subquery returning MAX price for Accessories. Return product_name, category, and price ordered by

price descending.*/

```
select product_name, category, price from products
where price > (select max(price) from products
where category = 'accessories')
order by price desc;
```

-- Q20. Using an IN subquery, find all orders placed by customers who joined in the year 2021. Return order_id, customer_id, amount, and order_date, ordered by order_date.

```
select order_id, customer_id, amount, order_date from orders
where customer_id in (select customer_id from customers
where join_year = 2021)
order by order_date;
```

-- Q21. Using EXISTS, find all employees who are managers (i.e., at least one other employee has their emp_id as manager_id). Return emp_id, emp_name, and department, ordered by emp_name.

```
select emp_id, emp_name, department from employees e1
where exists (select 1 from employees e2
where e2.manager_id = e1.emp_id)
order by emp_name;
```

-- Q22. Using NOT EXISTS, find all employees who are NOT managers — meaning no other employee lists them as their manager_id. Return emp_id, emp_name, and department, ordered by emp_id.

```
select emp_id, emp_name, department from employees e1
where not exists (select 1 from employees e2
where e2.manager_id = e1.emp_id)
order by emp_id;
```

-- Q23. Using EXISTS with a correlated subquery on the orders table, find all customers who have placed at least one order. Return customer_id, customer_name, and city, ordered by customer_id.

```
select customer_id, customer_name, city from customers c
where exists (select 1 from orders o
where o.customer_id = c.customer_id)
order by customer_id;
```

-- Q24. Using NOT EXISTS, find all customers who have never placed any order. Return customer_id, customer_name, and city, ordered by customer_id.

```
select customer_id, customer_name, city from customers c
where not exists (select 1 from orders o
where o.customer_id = c.customer_id)
order by customer_id;
```

-- Q25. Using EXISTS with a correlated subquery on the orders table, find all products that appear in at least one order. Return product_id, product_name, and category, ordered by product_id.

```
select product_id, product_name, category from products p
where exists (select 1 from orders o
where o.product_id = p.product_id)
order by product_id;
```

/*

-- Q26. Find all employees (excluding emp_id 105 — Eva Green herself) who work in the same department as emp_id

105. Use a single-row subquery to get Eva Green's department. Return emp_name, department, and salary ordered

by salary descending. */

```
select emp_name, department, salary from employees
where department = (select department from employees where emp_id = 105)
and emp_id <> 105
order by salary desc;
```

/*

-- Q27. Find the order_id, customer_id, amount, and order_date of the most recently placed order (the order with

the latest order_date in the table). Use a single-row subquery returning MAX(order_date). */

```
select order_id, customer_id, amount, order_date from orders
where order_date = (select max(order_date) from orders);
```

-- Q28. Find the order_id, customer_id, amount, and order_date of the earliest order ever placed (the order with the minimum order_date). Use a single-row subquery.

```
select order_id, customer_id, amount, order_date from orders
where order_date = (select min(order_date) from orders);
```

/*

-- Q29. Find all employees whose salary is greater than the salary of EVERY employee in the HR department. In other

words, their salary must exceed even the highest-paid HR employee. Return emp_name, department, and salary

ordered by salary descending. (Use a scalar subquery returning MAX salary of HR.)*/*

```
select emp_name, department, salary from employees
where salary > (select max(salary) from employees
where department = 'HR')
order by salary desc;
```

/*

-- Q30. Among orders with status = 'Delivered', find those whose amount is greater than the average amount of all

Delivered orders. Use a single-row scalar subquery to compute the average of Delivered orders. Return order_id,

customer_id, amount, and status ordered by amount descending.*/*

```
select order_id, customer_id, amount, status from orders
where status = 'delivered' and amount > (select avg(amount) from orders where status = 'delivered')
order by amount desc;
```

/*

Q31. Using a correlated subquery, find all employees who earn more than the average salary of their own

department. For each qualifying employee, also display their department's average salary (rounded to 2 decimal

places). Order by department, then salary descending. */

```
select e.emp_name, e.department, e.salary,  
round((select avg(e2.salary) from employees e2  
where e2.department = e.department), 2) as dept_avg from employees e  
where e.salary > (  
select avg(e3.salary) from employees e3  
where e3.department = e.department)  
order by e.department, e.salary desc;
```

/*

Q32. For every employee, use a correlated subquery to count how many other employees in the same department

earn strictly more than them. Display emp_name, department, salary, and this count as 'higher_earners'. Order by

department, salary descending. */

```
select e.emp_name, e.department, e.salary,  
(select COUNT(*) from employees e2  
where e2.department = e.department and e2.salary > e.salary) as higher_earners  
from employees e  
order by e.department, e.salary desc;
```

/*

Q33. Using a correlated subquery inside the WHERE clause, find all customers whose total order spend (sum of all

their order amounts) is greater than the average total spend per customer across all ordering customers. Display

customer_id, customer_name, and total_spend. Order by total_spend descending. (Compute per-customer total and

the overall average both via subqueries — no window functions.)

*/

```
select c.customer_id, c.customer_name,  
(select sum(o1.amount) from orders o1  
where o1.customer_id = c.customer_id) as total_spend  
from customers c where (select SUM(o2.amount)  
from orders o2  
where o2.customer_id = c.customer_id) > (select avg(customer_total)  
from (select sum(amount) as customer_total  
from orders  
group by customer_id) as totals)  
order by total_spend desc;
```

/*

Q34. Using a subquery in the WHERE clause with IN and a GROUP BY / HAVING inside the subquery, find all

employees who belong to departments where the average salary of the department is greater than 65,000. Return

emp_name, department, and salary, ordered by department and salary descending. */

```
select emp_name, department, salary  
from employees  
where department in (select department  
from employees  
group by department  
having avg(salary) > 65000)  
order by department, salary desc;
```

/*

Q35. For every row in the orders table, use a correlated subquery inside a CASE expression to label each order as

'Above Avg' if its amount is greater than or equal to that customer's own average order amount, and 'Below Avg'

otherwise. Show order_id, customer_id, amount, and the label as 'vs_cust_avg'. Order by customer_id, order_id.

*/

```
select o.order_id, o.customer_id, o.amount,
case
    when o.amount >= (select avg(o2.amount)
    from orders o2
    where o2.customer_id = o.customer_id)
    then 'Above Avg'
    else 'Below Avg'
end as vs_cust_avg
from orders o
order by o.customer_id, o.order_id;
```

/*

Q36. Using a correlated subquery, find all products whose price is greater than the average price of other products

in the same category. Display product_name, category, price, and the category average (rounded to 2 decimals) as

'cat_avg'. Order by category, price descending.

*/

```
select p.product_name, p.category, p.price,
round((select avg(p2.price)
from products p2
where p2.category = p.category
and p2.product_id <> p.product_id), 2) as cat_avg
from products p
```



```
where p.price > (select avg(p3.price)
from products p3
where p3.category = p.category
and p3.product_id <> p.product_id)
order by p.category, p.price desc;
```

/*

Q37. Find all employees whose salary is greater than the salary of AT LEAST ONE employee in the Marketing

department. Use ANY (or equivalently, > MIN of Marketing salaries). Return emp_name, department, and salary

ordered by salary descending. Note: Ivy Turner (56,000) is the lowest-paid Marketing employee — so all employees

earning above 56,000 qualify.

*/

```
select emp_name, department, salary
from employees
where salary > any (select salary
from employees
where department = 'Marketing')
order by salary desc;
```

/*

Q38. Find all employees whose salary is less than the salary of EVERY employee in the Finance department. In

other words, their salary must be below even the minimum Finance salary (70,000). Use ALL (or < MIN of Finance).

Return emp_name, department, and salary ordered by salary descending.

*/

```
select emp_name, department, salary
from employees
```

```
where salary < all (select salary
from employees
where department = 'finance')
order by salary desc;
```

```
/*
```

Q39. Find customers who have placed at least one order AND whose every order has status = 'Delivered' (i.e., they have no Cancelled or Pending orders). Use a combination of EXISTS and NOT EXISTS. Return customer_id and customer_name ordered by customer_id.

```
*/
```

```
select customer_id, customer_name
from customers c
where exists (select 1 from orders o
where o.customer_id = c.customer_id)
and not exists (select 1
from orders o2
where o2.customer_id = c.customer_id
and o2.status <> 'delivered')
order by customer_id;
```

```
/*
```

Q40. Using nested IN subqueries, find all products that were ordered by at least one customer from the city of Mumbai. First find customer_ids from Mumbai, then find product_ids ordered by those customers, then return the product details. Show product_id, product_name, and category, ordered by product_id.*/

```
select product_id, product_name, category
from products
```

```
where product_id in (select product_id from orders
where customer_id in (select customer_id
from customers
where city = 'mumbai'))
order by product_id;
```

```
/*
```

Q41. Find the employee(s) with the second-highest salary in the entire company. Use a subquery that first finds the

maximum salary, then an outer query that finds the maximum salary below that value. Return emp_name, department, and salary.*/

```
select emp_name, department, salary
from employees
where salary = (select max(salary) from employees
where salary < (select max(salary) from employees));
```

```
/*
```

Q42. Using NOT EXISTS, find all departments where every single employee earns strictly more than 50,000. (i.e.,

there is no employee in that department with salary <= 50,000.) Return just the department name, ordered

alphabetically. Note: Leo Harris in HR earns exactly 50,000, so HR does not qualify. */

```
select distinct e1.department
from employees e1
where not exists (select 1 from employees e2
where e2.department = e1.department
and e2.salary <= 50000)
order by e1.department;
```

/*

Q43. Using a correlated subquery in the WHERE clause, find all customers who have placed orders for at least 2

different products (i.e., their orders contain at least 2 distinct product_id values). Return customer_id and

customer_name, ordered by customer_id. */

```
select c.customer_id, c.customer_name
```

```
from customers c
```

```
where (select count(distinct o.product_id)
```

```
from orders o
```

```
where o.customer_id = c.customer_id) >= 2
```

```
order by c.customer_id;
```

/*

Q44. Using a correlated subquery, find all employees whose salary is above the average salary of all employees

hired in the same year as them. Show emp_name, department, salary, hire_year, and the year's average (rounded to

2 dp) as 'year_avg'. Order by hire_year, salary descending. */

```
select e.emp_name, e.department, e.salary, e.hire_year,
```

```
round((select avg(e2.salary)
```

```
from employees e2
```

```
where e2.hire_year = e.hire_year), 2) as year_avg
```

```
from employees e
```

```
where e.salary > (select avg(e3.salary)
```

```
from employees e3
```

```
where e3.hire_year = e.hire_year)
```

```
order by e.hire_year, e.salary desc;
```

/*

Q45. Using a correlated subquery, find all products whose stock_qty is less than the average stock_qty of all

products in the same category. Display product_name, category, stock_qty, and the category average stock rounded

to 2 dp as 'cat_avg_stock'. Order by category, stock_qty ascending. */

```
select p.product_name, p.category, p.stock_qty,  
round((select avg(p2.stock_qty)  
from products p2  
where p2.category = p.category), 2) as cat_avg_stock  
from products p  
where p.stock_qty < (select avg(p3.stock_qty)  
from products p3  
where p3.category = p.category)  
order by p.category, p.stock_qty asc;
```

/*

Q46. Find all employees whose salary is among the top 3 distinct salary values in the company. For example, if the

top 3 distinct salaries are 82000, 80000, and 78000, return all employees earning any of those values. Use a subquery

with LIMIT or a nested approach — no window functions.*/

```
select emp_name, department, salary  
from employees  
where salary in (select distinct salary  
from employees  
order by salary desc  
limit 3  
)  
order by salary desc;
```

/*Q47. Find all customers (who have placed at least one order) where every single one of their orders has an amount

strictly greater than 1,000. Customers with any order at or below 1,000 must be excluded. Use EXISTS and NOT

EXISTS. Return customer_id and customer_name, ordered by customer_id.*/

```
select customer_id, customer_name
from customers c
where exists (select 1 from orders o
where o.customer_id = c.customer_id)
and not exists (select 1 from orders o2
where o2.customer_id = c.customer_id
and o2.amount <= 1000)
order by customer_id;
```

/*Q48. For each department, find the employee whose salary is closest to that department's average salary

(minimum absolute difference). If two employees are equally close, both should appear. Show emp_name,

department, salary, dept_avg (rounded to 2 dp), and the difference (diff). Order by department. */

```
select e.emp_name, e.department, e.salary,
round((select avg(e2.salary) from employees e2
where e2.department = e.department), 2) as dept_avg,
abs(e.salary - (select avg(e3.salary) from employees e3
where e3.department = e.department)) as diff
from employees e
where abs(e.salary - (select avg(e4.salary) from employees e4
where e4.department = e.department)) = (
select min(abs(e5.salary - (select avg(e6.salary)
from employees e6
where e6.department = e5.department)))
```

```
from employees e5
where e5.department = e.department)
order by e.department;
```

/*Q49. Find all customers (other than customer 201) who have ordered every product that customer 201 has

ordered. Customer 201 (Aarav Mehta) has ordered product_ids 301, 302, and 304. A candidate customer must have

ordered all three. Use a NOT EXISTS / EXCEPT approach. Return customer_id and customer_name ordered by

customer_id. (With this dataset, the expected result is an empty set.) */

```
select c.customer_id, c.customer_name from customers c
where c.customer_id <> 201
and not exists (select 1 from orders o201
where o201.customer_id = 201
and not exists (select 1 from orders oc
where oc.customer_id = c.customer_id
and oc.product_id = o201.product_id))
order by c.customer_id;
```

/*Q50. Find all departments whose total salary bill is greater than the total salary bill of at least one other

department (i.e., greater than the minimum department total). Show department and dept_total ordered by

dept_total descending. Use a subquery in the HAVING clause. */

```
select department, sum(salary) as dept_total
from employees
group by department
having sum(salary) > ( select min(dept_total)
from (select sum(salary) as dept_total
from employees
```

```
group by department
) as dept_totals)
order by dept_total desc;
```

/*Q51. Using only subqueries (no self-join syntax), find all employees who earn strictly more than their own

manager's salary. For each such employee, display emp_name, their salary as emp_salary, their manager's name as

manager_name, and the manager's salary as manager_salary. Order by emp_salary descending. */

```
select e.emp_name, e.salary as emp_salary,
(select m.emp_name from employees m
where m.emp_id = e.manager_id) as manager_name,
(select m.salary from employees m
where m.emp_id = e.manager_id) as manager_salary
from employees e
where e.manager_id is not null
and e.salary > (
select m.salary from employees m
where m.emp_id = e.manager_id)
order by emp_salary desc;
```

/*Q52. Find all products whose total number of orders is greater than the average number of orders per product

(across only those products that appear in at least one order). Use a correlated subquery to count each product's

orders, and a non-correlated subquery to compute the average order count per product. Show product_id,

product_name, and order_count. */

```
select p.product_id, p.product_name,
(select count(*) from orders o
where o.product_id = p.product_id) as order_count
from products p
where (select count(*) from orders o
```



```
where o.product_id = p.product_id) > (select avg(cnt)
from (select count(*) as cnt from orders
group by product_id) as product_counts);
```

/*Q53. Find the customer(s) whose single highest order amount is equal to the global maximum order amount in the

entire orders table. For each such customer, show customer_id, customer_name, and their best_order (maximum

order amount). Use correlated scalar subqueries — no joins.*/

```
select c.customer_id, c.customer_name,
(select max(o.amount) from orders o
where o.customer_id = c.customer_id) as best_order
from customers c
where (select max(o.amount)
from orders o
where o.customer_id = c.customer_id) = (
select max(amount)
from orders)
order by c.customer_id;
```

/*Q54. Find all employees who belong to departments that have a budget greater than the average budget across all

departments in the departments table. Use a subquery with IN to identify qualifying departments, then retrieve

employees. Show emp_name, department, and salary ordered by department and salary descending.*/

```
select emp_name, department, salary
from employees
where department in (select dept_name
from departments
where budget > (select avg(budget)
from departments))
```

order by department, salary desc;

/*Q55. For each customer who has placed at least one order, use a correlated subquery to count how many of their

own orders have an amount strictly greater than their personal average order amount. Display customer_id,

customer_name, and orders_above_avg. Order by orders_above_avg descending, then customer_id.
*/

```
select c.customer_id, c.customer_name,  
(select count(*) from orders o  
where o.customer_id = c.customer_id and o.amount > (  
select avg(o2.amount) from orders o2  
where o2.customer_id = c.customer_id)) as orders_above_avg  
from customers c where exists (select 1 from orders o3  
where o3.customer_id = c.customer_id)  
order by orders_above_avg desc, c.customer_id;
```

/*Q56. Find all products that were ordered exclusively by customers from a single city (i.e., every customer who ever

ordered that product is from the same city). Only consider products that appear in the orders table. Show

product_id, product_name, and that single city as 'only_city'. With the current data, check whether any product

meets this criterion. */

```
select p.product_id, p.product_name,  
(select min(c.city) from customers c  
where c.customer_id in (select o.customer_id from orders o  
where o.product_id = p.product_id)) as only_city  
from products p  
where p.product_id in (select distinct product_id  
from orders)  
and (select count(distinct c2.city) from customers c2
```

```
where c2.customer_id in (select o2.customer_id
from orders o2
where o2.product_id = p.product_id)) = 1;
```

/*Q57. Find employees who are the sole highest earner in their department — meaning they earn the maximum

salary in their department AND no other employee in that department earns the same amount.

Show emp_name,

department, and salary ordered by salary descending. */

```
select e.emp_name, e.department, e.salary from employees e
where e.salary = (select max(e2.salary) from employees e2
where e2.department = e.department)
and (select count(*) from employees e3
where e3.department = e.department
and e3.salary = e.salary) = 1
order by e.salary desc;
```

/*Q58. Find customers whose total spend is more than double the average total spend per customer. Use a subquery

to compute each customer's total spend and another to compute the average of those totals. Show customer_id,

customer_name, and total_spend. With the current data, verify whether any customer meets this criterion.*/

```
select c.customer_id, c.customer_name,
(select sum(o.amount) from orders o
where o.customer_id = c.customer_id) as total_spend
from customers c
where (select sum(o.amount) from orders o
where o.customer_id = c.customer_id) > 2 * (
select avg(total_spend)
from (select sum(amount) AS total_spend
```

```
from orders

group by customer_id) as customer_totals)

order by total_spend desc;
```

/*Q59. Find the department that has the highest count of employees earning above the company-wide average

salary. Use a correlated subquery to count above-average earners per department, and return only the top

department. Show department and above_avg_count. */

```
select d.department,(select count(*)
from employees e
where e.department = d.department
and e.salary > (select avg(salary)
from employees)) as above_avg_count
from (select distinct department
from employees) as d
order by above_avg_count desc
limit 1;
```

/*Q60. Find all employees who are the highest earner in their own department (salary = dept MAX) but whose

overall salary rank in the company is beyond position 3 — meaning at least 3 other distinct salary values in the

company are higher than theirs. Show emp_name, department, and salary ordered by salary descending. */

```
select e.emp_name, e.department, e.salary
from employees e
where e.salary = (select max(e2.salary) from employees e2
where e2.department = e.department)
and (select count(distinct e3.salary)
from employees e3
```

```
where e3.salary > e.salary) >= 3
```

```
order by e.salary desc;
```